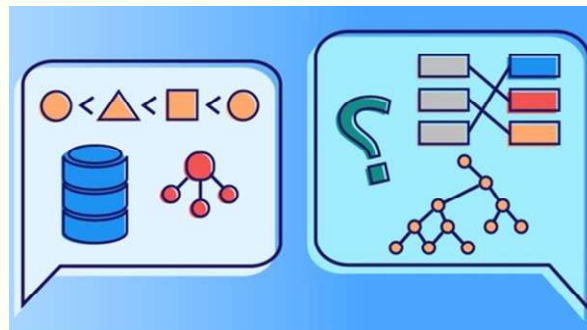


ALGORITMOS Y ESTRUCTURAS DE DATOS

Comisión F – 2026 – Cursado Anual



1

Algoritmos y Estructuras de Datos – AEDD – Calendario

Instancias de Evaluación	Parte	Fecha
Parcial 2	Parte Teórica Escrita	03/08/2026
	Parte Práctica Escrita	03/08/2026
	Parte Sitio Juez	03/08/2026
Parcial 2 - Recuperatorio	Parte Práctica Escrita	21/08/2026
	Parte Sitio Juez	17/08/2026
Parcial 3	<i>En el mes de septiembre (final) (fecha a confirmar)</i>	

2

Algoritmos y Estructuras de Datos – AEDD – Agenda 16/06

- Arreglos: Declaraciones.
- Arreglos: Dimensiones.
- Arreglos: Tamaño físico y tamaño real.
- Pasaje de arreglos a funciones.
- Borrar elementos en un arreglo.
- Insertar elementos en un arreglo.
- Buscar elementos en un arreglo:
 - Búsqueda secuencial.
 - Búsqueda binaria

3

Algoritmos y Estructuras de Datos – AEDD – Asistencia 16/06

Contraseña del estudiante
dshf5p

4

Estructuras de datos

- Simples o básicos: caracteres, booleanos, enteros, reales, flotantes.
- Estructurados: colección de valores relacionados. Se caracterizan por el tipo de dato de sus elementos, la forma de almacenamiento y la forma de acceso.
 - Estructuras estáticas: Su tamaño en memoria se mantiene inalterable durante la ejecución del programa, y ocupan posiciones fijas.

ARREGLOS - CADENAS - ESTRUCTURAS

- Estructuras dinámicas: Su tamaño varía durante el programa y no ocupan posiciones fijas.

LISTAS - PILAS - COLAS - ARBOLES - GRAFOS.

5

Estructuras de datos: Clasificaciones

- Según donde se almacenan
 - Internas** (en memoria principal)
 - Externas** (en memoria auxiliar)
- Según tipos de datos de sus componentes
 - Homogéneas** (todas del mismo tipo)
 - No homogéneas** (pueden ser de distinto tipo)
- Según la implementación
 - Provistas por los lenguajes** (básicas)
 - Abstractas** (TDA - Tipo de dato abstracto que puede implementarse de diferentes formas)
- Según la forma de almacenamiento
 - Estáticas** (ocupan posiciones fijas y su tamaño nunca varía durante todo el módulo)
 - Dinámicas** (su tamaño varía durante el módulo y sus posiciones también)

6

Estructura de datos: Arreglos

- Secuencia en memoria de un número determinado de **datos del mismo tipo** (**tipo base**: cualquier tipo simple o estructuras definidas por el usuario).
- Los datos se llaman **elementos** (celdas o componentes) del arreglo y se numeran consecutivamente 0,1,2,3,4, etc. Esos números se denominan **índice o subíndice del arreglo** y localizan la posición del elemento dentro del arreglo.
- Los elementos del arreglo se almacenan siempre en posiciones consecutivas de la memoria.
- Los elementos se **acceden en forma directa**, indicando el nombre del **arreglo y el subíndice** (variables indizadas).
- Pueden ser **unidimensionales** (se acceden con un solo índice) o **multidimensionales** (se acceden con varios índices).

7

Arreglos: Declaración

Los arreglos se deben declarar antes de utilizarse.

Formato de declaración:

tipo-de-dato nombre-del-arreglo [tamaño];

Por ejemplo, para crear un array de nombre *notas*, de diez variables enteras, se lo declara de la siguiente manera:

int notas [10];

El array *notas* contiene 10 elementos: el primero es *notas[0]* y el último es *notas[9]*.

8

Arreglos: Dimensiones

Un arreglo tiene tres partes:

- La dirección (ubicación en memoria de la primera variable indizada)
- El tipo base del arreglo (determina cuánta memoria –en bytes– ocupa cada variable indizada)
- El tamaño del arreglo (cantidad de elementos del arreglo)

Importante: C++ no comprueba que los índices estén dentro del rango definido.

¡Es responsabilidad del programador!

9

Arreglos: Tamaño físico y Tamaño real

En muchos problemas, pasa que al compilar se conozca el tamaño máximo que puede tener un arreglo, pero no el tamaño real, el que se conocerá en tiempo de ejecución y que incluso puede variar de un momento a otro.

Es necesario entonces definir una variable que contenga en todo momento el tamaño actual del arreglo (subíndice del último elemento cargado).

- Tamaño Físico: cantidad máxima de elementos que tiene un arreglo.
- Tamaño Real: cantidad exacta de valores almacenados dentro del arreglo.
 1. También se lo conoce como tamaño actual, tamaño lógico o tope lógico.
 2. Puede ser menor o igual al tamaño físico (pero nunca mayor).

En arreglos subocupados, los recorridos del arreglo deben realizarse teniendo como tope al tamaño real.

10

Arreglos: Pasaje como parámetros a funciones

Ejemplo de prototipo: `void funcion( int a[], int tam );`

El parámetro formal `int a[]` es un parámetro de arreglo.

Los corchetes sin expresión de índice dentro de los corchetes son lo que C++ usa para indicar un parámetro de arreglo. Cualquier valor que se coloque entre los corchetes será ignorado.

Un parámetro de arreglo no efectúa una copia del contenido del arreglo, sino que copia la referencia a la dirección de memoria en la que empieza el arreglo.

Cuando usamos un arreglo como argumento en una llamada de función, cualquier acción que se efectúe con el parámetro de arreglo se efectúa sobre el argumento arreglo, así que la función puede modificar los valores de las variables indizadas del argumento arreglo.

11

Pasaje de arreglo a funciones

Supongamos el prototipo: `void funcion( int a[], int tam );`

Si tenemos las siguientes declaraciones:

`int a[5], numero_de_elementos = 5;`

Una posible llamada a la función `funcion` podría ser:

`funcion(a, numero_de_elementos);`

El argumento `a` dado en la llamada a la función `funcion` se da sin corchetes ni expresión de índice.

El argumento arreglo no le indica a la función qué tamaño tiene el arreglo. Por ello, resulta indispensable incluir siempre otro argumento de tipo `int` que indique dicho tamaño.

Parámetro de arreglo

Tamaño del arreglo

12

Pasaje de arreglo a funciones como constante

Podemos indicar que no queremos modificar el argumento arreglo, y entonces la computadora se asegurará de que el código no modifique inadvertidamente algún valor del arreglo.

```
void mostrar(const int a[], int tamano_de_a)
{
    cout << "El arreglo contiene los siguientes valores:\n";
    for (int i = 0; i < tamano_de_a; i++)
        cout << a[i] << " ";
    cout << endl;
}
```

La siguiente sentencia será detectada como error:

```
for ( int i = 0; i < tamano_de_a; a[i]++ )
```

Un parámetro de arreglo modificado con **const** es un parámetro de arreglo constante.

13

Algoritmos y Estructuras de Datos – AEDD – Asistencia 16/06

Contraseña del estudiante
dshf5p

14

¡¡Resolvamos un ejercicio! *La Secuencia del Oráculo*

Una antigua civilización dejó grabada una secuencia de números misteriosos en una piedra. Se cree que esta secuencia puede predecir eventos futuros, pero para interpretarla correctamente, debe analizarse de forma especial.

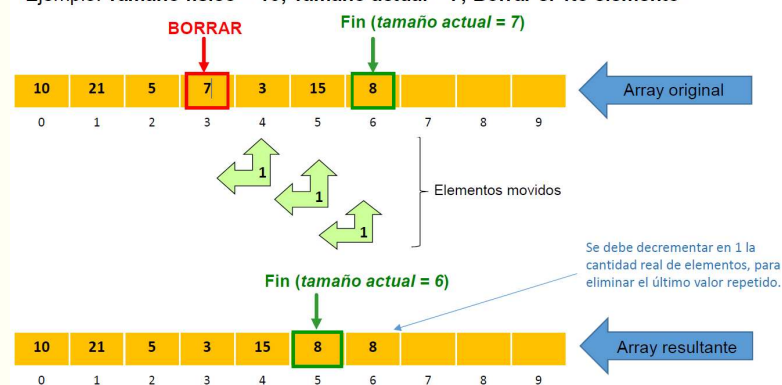
Se te entrega una secuencia de N números enteros positivos. Se dice que el "oráculo" es capaz de detectar el número de picos en la secuencia. Un pico es un elemento que es mayor que sus vecinos adyacentes (excepto si está en los extremos, en cuyo caso no se considera). Debes leer la secuencia de números en un arreglo unidimensional y luego implementar una función recursiva que cuente cuántos picos hay en la secuencia y luego mostrar el resultado final.

15

Borrar un elemento de un arreglo

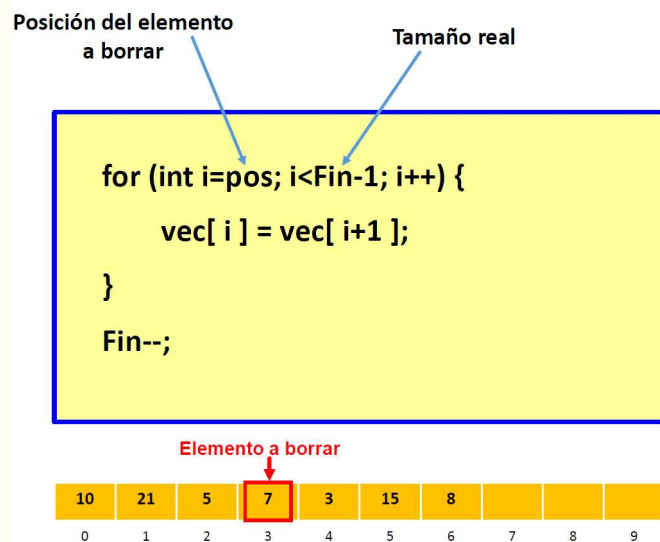
Implica correr los elementos que estén a la derecha del elemento a borrar, 1 posición hacia la izquierda, comenzando por el más cercano a dicho elemento. Además, decrementar en 1 la cantidad real (tamaño lógico o actual) de elementos.

■ Ejemplo: Tamaño físico = 10; Tamaño actual = 7; Borrar el 4to elemento



16

Borrar un elemento de un arreglo



17

¡¡Resolvamos un ejercicio! *Borrar un elemento*

Solicitar el tamaño real de un vector < 100, luego solicitar los números enteros de cada posición.

Luego solicitar la posición del elemento a borrar.

Mostrar los elementos del vector, luego del borrado.

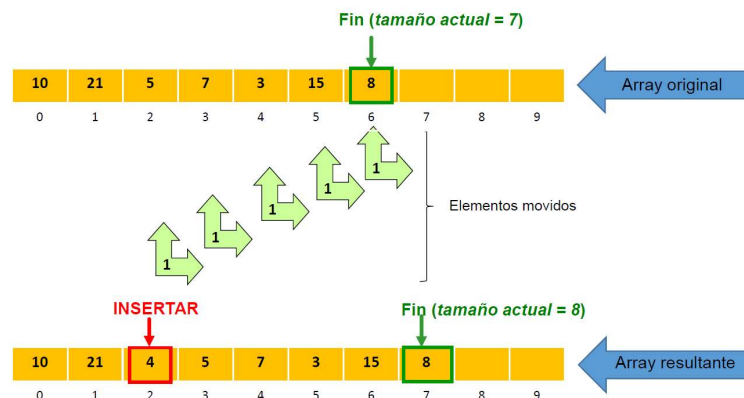
¿Y si tenemos que borrar un rango de valores?

18

Insertar un elemento en un arreglo

Implica correr los elementos 1 posición hacia la derecha, a partir de la última posición y hasta la posición en la que se va a insertar. Además, incrementar en 1 la cantidad real (tamaño lógico o actual) de elementos.

■ Ejemplo: Tamaño físico = 10; Tamaño actual = 7; Insertar 4 en posición 2



19

Insertar un elemento en un arreglo

Tamaño real

Posición donde se insertará el elemento

```
for (int i=TL-1; i>=pos; i--) {
    vec[ i+1 ] = vec[ i ];
}
vec[ pos ]=valor; //se inserta el elemento
TL++;
```

Insertar 4 en posición 2



20

Buscar un elemento en un arreglo

El algoritmo difiere si el arreglo está o no ordenado por algún criterio (ascendente o descendente).

- Si no hay orden, deben examinarse los elementos de izquierda a derecha (o de derecha a izquierda) hasta encontrar el elemento buscado o hasta que se hayan examinado todos los elementos → *Búsqueda Secuencial*.
- Si hay orden, debe examinarse el elemento central del arreglo, si el elemento a buscar es menor que éste, se buscará en la primera mitad del arreglo, sino se buscará en la segunda mitad. Se repite esta acción hasta que se encuentre el elemento o hasta que se obtenga una mitad consistente de un solo elemento que no es el buscado → *Búsqueda Binaria*.

21

Buscar un elemento en un arreglo

Buscar un elemento en un arreglo: Dados un arreglo $A[N]$ y un elemento X , determinar si existe algún i , tal que $0 \leq i < N$ y $A[i]=X$

Cuando A ni esta ordenado, se realiza una *Búsqueda Secuencial*.

```
int i=0;

while (i<N and A[i]!=X){
    i++;
}

if(i<N)
    cout << "X encontrado en posicion " << i;
else
    cout << "X no encontrado";
```

- Mejor caso: 0 pasadas del ciclo while
- Peor caso: N pasadas del ciclo while

22

Buscar un elemento en un arreglo

Alternativa usando for: ¡Opción no válida para la cátedra!

- Ejemplo: Buscar un elemento en un arreglo usando **Búsqueda Lineal**

```
//La funcion devuelve la ubicacion de la clave en el arreglo
//Se devuelve un -1 si no se encuentra el valor
int busquedaLineal(int vec[], int tamaño, int clave){
    int i;

    for (i = 0; i < tamaño; i++){
        if (vec[i] == clave)
            return i;
    }
    return -1;
}
```

El return hace que el ciclo FOR se interrumpa!

23

Buscar un elemento en un arreglo

Buscar un elemento en un arreglo: Dados un arreglo $A[N]$ y un elemento X , determinar si existe algún i , tal que $0 \leq i < N$ y $A[i] = X$

Si A está ordenado: **Búsqueda Binaria**: se examina el elemento central, si no es el buscado, se determina en qué mitad (izquierda o derecha) puede estar el elemento buscado y se pasa a examinar esa mitad, repitiendo este criterio hasta encontrar el valor buscado o hasta que el intervalo de búsqueda sea menor que 1.

```
izq=0; der=N-1;
med = (izq + der)/2;
while (izq <= der and A[med] != X) {
    if (X < A[med])
        der = med-1;
    else
        izq=med+1;

    med = (izq + der)/2;
}

if (der < izq)
    cout << "X no encontrado";
else
    cout << "X encontrado en posición " << med;
```

- Mejor caso: 0 pasadas del ciclo while
- Peor caso: $\log_2 N$ pasadas del ciclo while

24

Buscar un elemento en un arreglo ¡Tarea!

Buscar un elemento en un arreglo: Dados un arreglo $A[N]$ y un elemento X , determinar si existe algún i , tal que $0 \leq i < N$ y $A[i] = X$

Implementar la función de búsqueda binaria de un elemento en una lista. Devuelve la posición i del elemento X en el arreglo A . Si no esta devuelve -1.

- Mejor caso: 0 pasadas del ciclo while
- Peor caso: $\log_2 N$ pasadas del ciclo while

25

Algoritmos y Estructuras de Datos – AEDD – Asistencia 16/06

Contraseña del estudiante

dshf5p

26

Algoritmos y Estructuras de Datos – AEDD – Agenda 16/06

- ✓ Arreglos: Declaraciones.
- ✓ Arreglos: Dimensiones.
- ✓ Arreglos: Tamaño físico y tamaño real.
- ✓ Pasaje de arreglos a funciones.
- ✓ Borrar elementos en un arreglo.
- ✓ Insertar elementos en un arreglo.
- ✓ Buscar elementos en un arreglo:
 - ✓ Búsqueda secuencial.
 - ✓ Búsqueda binaria

¡¡Objetivo cumplido!!!

27

AEDD – Otras tareas !!!!!

Para profundizar en el tema:

Libro:

Resolución de problemas con C++ - Savitch

Capítulo 10

Libro:

C++ para ingeniería y ciencias - Gary J. Bronson

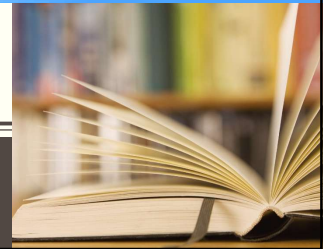
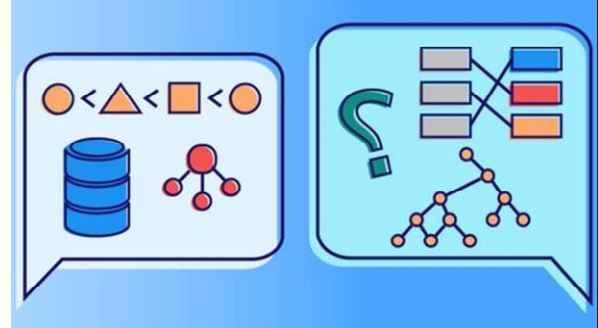
Capítulo 11

28

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual

¿DUDAS?



29

ALGORITMOS Y ESTRUCTURA DE DATOS

Comisión F – 2026 – Cursado Anual



30